OEIS Program Synthesis, Verified Semantics, and Graded Similarity: A Unified Account

Oruži Zarathustra Goertzel

July 21, 2026

Abstract

We collect an ongoing line of work built around the On-Line Encyclopedia of Integer Sequences (OEIS) as a testbed for verified program synthesis. Three strands meet here. First, a small domain-specific language (DSL) of integer-sequence programs is given a machine-checked operational semantics in Lean, together with a token *recognizer*, a big-step generative-semantics (GSLT) layer with modal emission specifications, and a family of formal *obligations* that a synthesizer must respect. Second, a neural sequence-to-graph synthesizer is organized so that the network's operational contracts—a legality mask, a canonical form, relabeling invariance—have machine-checked formal counterparts. Third, a graded similarity layer expressed in probabilistic-logic truth values $\langle \text{strength}, \text{confidence} \rangle$ recovers structure that an exact deduplication hash discards: it audits train/test leakage, while a conservative abstract interpreter proposes totality, sign, and parity invariants for each program. We report concrete measurements over a corpus of 104,083 synthesized programs, distinguish empirical validation of that interpreter from kernel proof, and show how verified properties could safely guide search and generate proof obligations.

Open artifacts. The formal repository, paper source, compiled PDF, and neural experiment repository are public. The complete 33-program adjudication module and the provenance-aware program-discovery development are linked separately. The theorem links below are immutable GitHub source permalinks to commit `26dd10a61a969abea0bbae023e70174491e2a946`; they open the complete statement and proof, not a search page or a moving branch location.

1 Introduction

The OEIS [3] is a natural benchmark for program synthesis: each entry is an integer sequence, and a solution is a program that reproduces its terms. Recent self-learning systems synthesize such programs at scale from a small combinator DSL, discovering programs for a large fraction of the OEIS without human-written training data [1, 2]. Our contribution is not another synthesizer but a *verified and measured* account of the objects a synthesizer manipulates.

We separate three concerns that are often conflated:

- **What a program *is*** — its syntax over a fixed operator vocabulary, and the machine-checked semantics of running it (Sections 2–3).
- **What a program *does on a probe*** — its input/output behaviour on a finite window (its *signature* σ), a finite observational summary (Section 5).

operator	arity	role	partiality / termination
`zero`,`one`,`two`	0	constants	total
`x`,`y`	0	variables ($n \geq 0$; inner ≥ 0)	total
`addi`,`diff`,`mult`	2	arithmetic	total (resource: overflow)
`divi`,`modu`	2	division / modulo	*genuine*: error on zero divisor
`cond`	3	branch on ≤ 0	total given branches
`loop`,`loop2`	3, 5	bounded iteration	terminating (resource: timeout)
`compr`	2	unbounded search	*genuine*: may not terminate
`push`	2	stack push	total (resource: push limit)
`pop`	1	stack pop	*genuine*: error on empty

Table 1: The synthesis DSL. Only `divi`/`modu` (zero divisor), `pop` (empty stack), and `compr` (non-terminating search) introduce *genuine* partiality; `loop`/`loop2`, overflow, and timeouts are *resource* limits. This three-way character drives the totality analysis.

- **What properties static analysis can certify** — totality, sign, and parity: intensional information inferred conservatively rather than read off finitely many outputs (Section 6).

The payoff of keeping these apart is a clean division of labour: the probe layer gives cheap graded evidence but can never certify behaviour beyond the probe window; the intensional layer proposes conservative all-input certificates, while the Lean layer can turn selected claims into kernel proofs. Combined, they sharpen both benchmark hygiene and the discovery of conjectures.

2 The OEIS synthesis DSL

Programs are postfix token streams over a 16-operator vocabulary evaluated on a two-register integer state. The operators are constants `zero`, `one`, `two`; the index variable `x` $= n$ and an inner variable `y`; binary arithmetic `addi`, `diff`, `mult`, `divi`, `modu`; a conditional `cond` (test ≤ 0); bounded iterators `loop` and `loop2`; an unbounded search `compr` (the a-th $n \geq 0$ satisfying a predicate); and stack operators `push`, `pop`. Table 1 summarizes the arities and the partiality character of each operator, which is decisive for the analysis of Section 6.

A closed form illustrates the language: the factorial $n!$ is `loop(mult(x,y),x,one)` — fold multiplication by the running counter n times starting from 1. The counting sequence $a(n) = n+1$ is `addi(x,one)`.

3 Verified GSLT semantics

The DSL is given a machine-checked operational semantics in Lean. On top of it we build a generative-semantics layer as a triple (T, E, R): terms T, an equational theory E (a setoid), and a rewrite relation R with left/right coherence laws. The coarse *big-step GSLT* rewrites a program to the integer value its fuel-bounded evaluator emits; a completed-value term is never reflexively a program, so non-triviality is load-bearing. Two modalities express synthesis goals:

$$\Diamond(\text{emits } v \text{ at } k) \iff \exists\, s',\ \texttt{eval}\ \textit{fuel}\ \textit{sig}\ p\ (\texttt{seed}\ k)\ = \texttt{some}(v, s'),$$
$$\texttt{EmitsPrefix}\ p\ \ell \iff \forall i\, v,\ \ell_i = v \Rightarrow \Diamond(\text{emits } v \text{ at } i).$$

`EmitsPrefix` is precisely the OEIS synthesis specification: "p reproduces the observed prefix ℓ."

Obligations. A synthesizer that claims correctness must respect a family of formal obligations. We single out three that are discharged as machine-checked theorems, axiom-clean (each theorem's `#print axioms` footprint lies within {propext, Classical.choice, Quot.sound}):

Obligation 1 (Recognizer completeness / mask equivalence)**.** *The legality predicate* `maskLegal` *on token streams coincides with the executable recognizer (*`recognize_iff_maskLegal`*), and every well-formed program is recognized (*`recognize_complete`*). Hence a constrained decoder that emits only mask-legal tokens can generate exactly the well-formed programs.*

Obligation 2 (Relabeling invariance)**.** *Evaluation is invariant under a signature isomorphism (*`eval_relabel_signature_iso`*): renaming primitives consistently does not change the computed sequence. Behaviour depends on structure, not on operator names.*

Obligation 3 (Canonicalization soundness)**.** *Flag-driven canonicalization preserves semantics; the relevant proofs are* `canonicalize_sound` *and* `orgCanonicalize_sound`*. Thus, for example, sorting the children of commutative operators cannot make canonical-form deduplication merge behaviourally distinct programs.*

Further obligations include equational-theory soundness and evaluation congruence for the modalities. The machine-checked `finite_probe_rigidity_negative_witness` already settles finite-probe rigidity in the negative: two programs can agree on a finite probe while differing extensionally. This is the formal shadow of the leakage phenomenon in Section 5. A separate small-step account remains future work; the present rewrite layer deliberately uses the evaluator as its big-step relation.

4 The synthesizer as specification

The neural synthesizer maps a sequence prefix to a program graph (sequence-to-graph). Its decoder is *execution-guided*: at each step a legality mask restricts the token set to those that can extend a well-formed program, and the mask is exactly the recognizer of Obligation 1. Successive training variants add retrieval and contrastive objectives that consult a similarity index. The central design stance is that the network's operational contracts are the prover's theorems: the mask's soundness, the canonical form's soundness, and relabeling invariance are not engineering conveniences but the denotational safety envelope of the synthesizer.

5 Leakage-safe data and the probe-behaviour layer

Exact deduplication and splits. Each program's probe signature σ is its output vector on $n = 0, \ldots, 31$ at a fixed fuel, with $\perp$ (`None`) marking a non-value result at that budget. Programs with identical σ form an exact *probe-signature* class (called an "E-class" in the implementation); this is not extensional program equivalence. The train/val/test split is assigned by whole connected component so that no exact-signature duplicate straddles the split boundary. Over 104,083 programs this gate is leak-tight for the measured signatures: all 3,007 exact-σ duplicate pairs lie in the same split.

Graded similarity. Exact equality is brittle. We express graded similarity as a probabilistic-logic truth value $\langle s, c \rangle$ [6]: the probe-level strength is behavioural agreement over the *defined* domain, $s = |\text{agree}|/|\text{informative}|$, and confidence follows the chosen evidence-count law $c = m/(m + k)$,

strength band	cross-split pairs	train↔test pairs
0.80–0.90	17,250	6,365
0.90–0.95	2,614	909
0.95–0.99	502	150

Table 2: Soft leakage: near-equivalent program pairs ($d \geq 1$, not byte-identical) placed in different splits by the exact gate. Distinct evaluation programs with a train near-twin: 1.84% (test and val) at ≥ 0.95; 5.4–5.8% at ≥ 0.90.

with m the number of informative probe positions and a fixed evidence scale $k > 0$. This is an evidence-weighting convention, not an empirical calibration claim. For fully defined signatures, exact equality gives $s = 1$, $m = 32$; when $\perp$ occurs, m counts only informative positions. A locality-sensitive index over signatures generates near-equivalent candidate pairs without an $O(n^2)$ scan.

Soft-leakage audit. The graded layer surfaces *near*-duplicates the exact gate cannot see. Table 2 reports pairs whose two programs sit in different splits. The benchmark-integrity figure is the fraction of evaluation programs with a near behavioural twin in *train*: at strength ≥ 0.95, 1.84% of test and 1.84% of val programs (lower bounds—the index can miss a pair, never invent one). A disagreement-locus analysis separates genuine near-twins (disagreements confined to a single term, none in the tail) from "false friends" that agree only on a shared trivial tail; the trustworthy signal concentrates in the ≥ 0.95 band. This vindicates gating on *exact* full-signature equality rather than a similarity threshold, which would either over-merge on trivial tails or miss single-term twins.

6 The intensional layer: static properties and their proof boundary

Where the probe layer reads behaviour off finitely many outputs, the intensional layer uses abstract interpretation [4] to infer candidate all-input invariants. Its transfer functions are designed to over-approximate the concrete operators, and loops are handled by Kleene fixpoints in finite lattices. The current analyzer is implemented in Python and tested against the evaluator; its transfer functions have not yet been proved sound against the Lean semantics. We therefore distinguish its conservative static certificates from the kernel-checked program-correctness proofs of Section 9. We compute three properties.

Property 1 (Totality)**.** *A program is* classified total *when only resource limits can stop it: it contains neither* `compr` *nor* `pop`*, and every* `divi/modu` *divisor is certified nonzero. This is a conservative lower bound—"unknown" does not mean partial.*

Property 2 (Sign)**.** *A sign-domain analysis (with* $x = n \geq 0$*, and* $a \bmod b \geq 0$ *for* $b > 0$*) certifies the OEIS keyword* `nonn` *(nonnegativity) for many programs under its abstract rules.*

Property 3 (Parity)**.** *A* $\mathbb{Z}/2$ *analysis certifies "always even" / "always odd" under its abstract rules (e.g.* `mult` *by an even factor is even).*

Validation gates. Two independent finite checks test the implementation; they do not replace a soundness proof. A static gate compares every sign/parity claim against the real signatures: 0

static property	count	share
total	37,996	36.5%
nonnegative (`nonn`)	45,251	43.5%
positive	14,907	14.3%
even	2,725	2.6%
odd	2,926	2.8%

Table 3: Static-property distribution over 104,083 programs. Shares are lower bounds under the analyzer's intended conservative interpretation.

violations across all 104,083 programs ($\approx$ 3.3M value-checks). An empirical totality gate re-runs the real evaluator on 500 programs classified total to $n = 48$ (beyond the probe window): 0 genuine errors, only resource timeouts. Table 3 gives the distribution.

Disambiguating the signature. A $\perp$ in σ is ambiguous: is the program genuinely partial, or merely slow at the probe budget? Under the analyzer's intended soundness, a total classification attributes it to resource exhaustion. Of the 12,966 programs carrying a $\perp$, 2,345 are classified total, while 10,621 are not, with attributed cause: 6,539 `compr` (non-terminating search), 3,023 `pop` (empty list), 1,059 `divi`/`modu` (possible zero divisor). The intensional layer thereby partitions a signal that finite execution alone leaves ambiguous.

7 Filtering and measuring in the OEIS loop

Both roles suggested by Table 3 are useful inside a synthesis loop.

Filtering. An OEIS target often carries stated properties—for example its keywords (`nonn`, `sign`, `mult`) or invariants justified by its defining formula. A candidate can be pruned soundly only when two premises are established: the target invariant is itself justified, and the analyzer proves an incompatible candidate invariant. An "unknown" abstract result must remain legal; requiring every candidate to be *proved* nonnegative would be incomplete and could discard a correct program. Thus, before the analyzer is connected to the decoder, uncertain properties may rank candidates, while hard pruning must be restricted to proved contradictions. A future Lean soundness theorem for the analyzer would make that restricted semantic mask recall-preserving on its stated domain.

Measuring. The same properties are theoretical measurements of a program and of a sequence. Totality analysis distinguishes a possibly partial sequence from one classified as slow-to-compute but total; the sign/parity results are static-analysis counterparts of OEIS keywords, so annotations can be predicted and checked at scale. More finely, the graded truth values give each program pair an evidence-weighted similarity, and formal proofs can supply factive intensional features. Revision is licensed only for source-disjoint packets: a proof must not be counted again as independent support for the translation or observations on which it depends. The corpus thereby becomes not merely programs and outputs, but programs, outputs, provenance, and proofs.

8 The loop compounds: a three-seed cumulative replication

The construction above is only interesting if the loop it serves actually runs. It does. We report a prospectively registered replication of the self-learning regime of [1] using a typed-action decoder over the verified skeleton of Section 4: the network never emits program text, only *legal construction actions* over an exact state, and a deterministic checker decides acceptance.

The protocol is cumulative. Each of three independently seeded worlds—with random seeds controlling initialisation, minibatch order, dropout, and search tie-breaking; the data pool (79,249 rows) and the 240 search tasks are shared—trains on its accumulated verified corpus, searches under a fixed budget, checks every candidate against the whole OEIS, adds newly covered sequences to its corpus, and repeats. The endpoint is cumulative coverage C_g and its increment $D_g = C_g - C_{g-1}$; no comparison against held-back external data governs continuation, since in the deployed regime no such reservoir exists.

The shared initial corpus is not chosen freely: it is the reference system's own public bootstrap seed `sol0` [1], 3,771 sequences found by *random* program search with no learning, and the very origin from which that system subsequently grew its full published corpus through many iterations of the loop we are running here. Starting from that seed rather than from a mature corpus is deliberate, with three consequences. The origin is reproducible and learning-free, a fixed public artefact anyone can replay from. Our run is then the *same* self-learning process re-executed—not a fork of another system's answers—so a rediscovery is a genuine held-out generalisation within our own lineage rather than a lookup. And the base stays small enough that a world's own discoveries form a measurable share of the training signal instead of being drowned by it, which is what makes the loop's dynamics observable at all. Coverage per world:

world	initial	after gen. 1	after gen. 2
17	3,771	3,789	3,846
29	3,771	3,841	3,891
43	3,771	3,807	3,862

At generation three we registered, before any update, a three-way comparison of how the accumulated discoveries enter training: *uniform* replay (literal pooling, under which the 194 cumulative discoveries receive their natural 0.24% of sampling mass); *balanced* replay (25% discovery mass); and balanced replay with the auxiliary predictive heads disabled. All three arms continue from the same per-world parent checkpoint under identical update counts and search budgets, and are judged solely by new whole-OEIS A-numbers beyond the pooled 3,986 already known. New A-numbers, generation three:

setting	world 17	world 29	world 43	pooled
uniform replay	48	39	72	159
balanced replay, aux. off	65	155	88	308
balanced replay, aux. on	135	155	177	**467**

Every setting is positive in every world: nine of nine cells extend coverage, so the loop compounds rather than saturating at this scale. Two registered contrasts separate the mechanism. Balanced replay is worth +308 pooled over uniform—at literal pooling the model's own discoveries are numerically drowned by the base corpus, and the loop is starved of exactly the signal it exists to generate. The auxiliary heads are worth a further +159. The generation-three increment under the

selected setting exceeds the generation-two increment by roughly threefold, and the setting carried into generation four was fixed by the registered selection rule rather than chosen after inspection.

Two negative results bound the claim, and we record them because they materially limit its interpretation. First, an ablation asking whether a world's own discoveries beat an equal mass of *unused external certified examples* answers no, in all three worlds (162 versus 253 pooled): feedback submitted more candidates yet explored fewer distinct programs (5,540 versus 6,233), a diversity contraction rather than a fitting failure. That comparison is a legitimate ablation and a poor objective—the fresh arm is an oracle-advantaged comparator that does not exist in the deployed loop, where the corpus is whatever the loop has verified. Second, no arm in any generation solved its own assigned held-out target within budget. The evidence therefore concerns learned *discovery* guidance over the catalogue—the regime [1] operates in—and not target-conditioned synthesis, which remains open.

What the verified layer contributes here is attributability rather than yield. Because the legality mask is machine-checked, a discovery cannot be an artefact of a malformed candidate slipping past a hand-written filter; because ranking is provably recall-preserving, guidance changes search order and cannot manufacture or destroy solutions; and because every accepted program is checked against the whole corpus, the increments above are exact catalogue-coverage counts under that checker rather than estimates or full-domain correctness claims. The loop's growth and the loop's trustworthiness are established by different machinery, and both are needed.

8.1 The compounding claim, tested against its own control

The replication above shares a weakness with the regime it replicates: cumulative coverage rises, but training time rises with it, so growth and continued optimisation are confounded. We therefore ran the compounding question separately, on a faithful port of the reference system's own architecture—a 10,781,697-parameter scaled-Luong bidirectional-LSTM sequence-to-sequence model—under the reference system's own protocol, in which every generation is trained *from fresh initialisation* and knowledge is carried by the accumulated corpus rather than by the weights.

That protocol makes an exact control available. In two independently initialised worlds, one arm trains generation two on the seed corpus *augmented* with generation one's own verified discoveries; the paired control trains on the seed corpus *alone* with fresh initialisation. Both are compute-matched, decoded and whole-OEIS-checked identically, and scored against the same fixed baseline of 5,631 known sequences, so the sole difference between them is whether the model's own discoveries entered its training data. That difference is the loop.

world	discoveries fed back	seed corpus only
1	1,980	375
2	2,051	519

Feeding verified discoveries back finds roughly four to five times the new frontier that re-seeding on the same corpus finds, in both worlds, with the pre-registered ordering met in each. Every count carries a content-addressed job identity and was reproduced by an independent verification pass. Because both arms start from fresh weights, no part of this gap can be attributed to accumulated optimisation: the corpus alone carries it.

The contrast with the replication above is itself informative, and we state it plainly rather than let it pass as an unnoticed deviation. The reference implementation re-initialises its network every generation; the typed-action campaign of this section continues weights and optimiser state

across generations. The two designs answer different questions—one isolates the corpus, the other measures a lineage—and only the first can attribute compounding to feedback without qualification.

8.2 Eight generations, two worlds, four state carriers

The lineage design was then run to its registered endpoint: two independently initialised worlds, eight generations each, four matched-capacity decoder state carriers over the shared typed-action interface—a gated recurrent vector, a typed-slot workspace, and two belief-register variants (retention derived from the drift statistics versus learned). Every candidate was checked against the whole encyclopedia; all 64 cells passed independent verification. Distinct new sequences across everything: 984, from near-identical world totals (639 and 638, sharing only 293).

state carrier	distinct	found by no other	training cost
recurrent vector (GRU)	435	241	1.00×
typed workspace	383	171	2.85×
belief, derived retention	294	119	7.94×
belief, learned retention	244	138	3.53×

Three readings, in decreasing order of certainty. First, the yield ordering tracks the *decisiveness* ordering exactly—the recurrent vector had the lowest action entropy, the highest gold-action probability, and the fewest duplicate proposals—which is the mechanism the `equal_crossEntropy_strictly_ordered_expectedYield` theorem isolates under a fixed search budget. The broader fixed-budget scope and its counterexamples are collected by `budgetedSearchYield_crown`. Second, 669 of 984 sequences were found by exactly one state carrier, with pairwise overlaps of 8–16%: the carriers steer search into substantially different territory, so the union is worth far more than the best single arm, yet a fixed-split portfolio under one budget reached only 399 against the best arm's 435—complementarity is real and naive allocation fails to harvest it. Third, and most tentatively, the derived-retention belief arm (retaining $\approx 94\%$ of prior evidence per revision) out-yielded the learned-retention arm, which learned to discard roughly half its evidence per revision—the direction the drift theory predicts in a near-stationary domain, where the optimal retention approaches one. Margins varied across the two worlds (one a clear recurrent-vector win, the other a near-tie), so these are descriptive statements about two runs, not population claims. Roughly three quarters of every arm's search budget was spent re-proposing programs it had already generated, which bounds how much any architectural conclusion can matter before search-side deduplication is addressed.

9 From coverage to proof: adjudicating discoveries against their definitions

Everything above establishes that a discovered program reproduces a sequence's stored terms. That is weaker than it sounds. The published entries are short—median 38 terms corpus-wide, and only 26 for the sequences our own loop discovered, with 22% resting on fewer than twenty terms and one on three. This gap is consistent with a simple selection mechanism: short entries impose fewer observed constraints and are therefore more vulnerable to coincidental matches. And finite agreement is exactly what our finite-probe rigidity counterexample proves insufficient—agreement on a finite prefix need not force extensional equality.

So we asked the question the loop cannot answer by itself: does the discovered program actually compute the sequence its entry *defines*? This is answerable, because the encyclopedia carries far

more than terms. Of our 984 discoveries, all 984 have keywords, 88% an explicit formula field, 88% a reference program, and 75% prose commentary. These fields provide candidate sources for specifications, but their precision varies: a closed formula is much less ambiguous than a keyword or informal comment.

Protocol. A definition can always be made to match a program after the fact, so the order of work is the whole methodology. Candidate programs were frozen first, from the completed campaign, as exact generated token streams with their hashes. Sequences were then selected on *metadata alone*, and each specification was formalized and locked *before* its candidate program was revealed. The specification therefore states a proof obligation rather than describing a known solution. This is the ordinary discipline of verification—a specification confronting a pre-existing implementation—and not, as one might worry, a translation from definition to program.

What is proved. For a specification with domain D, index map, and value function, and a frozen token stream t:

$$\text{Realizes}(\text{spec}, p) \;\equiv\; \forall\, i,\; D(\text{index}(i)) \;\to\; \text{EventuallyEmits}\big(p,\, i,\, \text{value}(\text{index}(i))\big),$$

where EventuallyEmits requires a fuel threshold beyond which the interpreter *stably* returns that value. Three things deserve emphasis. The quantifier is over the entire declared domain, not a probed window, so this is extensional correctness rather than extended testing. Termination is part of the claim: fuel stability is proved, not assumed, and a companion theorem establishes that a successful evaluation is unique across sufficient fuel (`eventuallyEmits_unique`). The formal definitions are `EventuallyEmits`, `Realizes`, and `CandidateRealizes`. Finally, the verified program is not a manual Lean reimplementation: the frozen token stream is parsed by the same recognizer the synthesizer uses, with the parse discharged by kernel computation, so what is verified is the artifact the network emitted.

Result. Thirty-three sequences were adjudicated, and every frozen program associated with them was proved extensionally correct on its full domain. Each carries an axiom audit; all depend only on the standard logical basis. Each record pins the OEIS identifier, entry revision, entry digest, offset, token stream and program hash, so a reader can reconstruct exactly which published text was translated. The registry cardinality is itself checked by `certifiedProofPacket_count`, and all 33 individual proofs are in the linked adjudication module above.

The OEIS definitions and metadata quoted below are attributed to the OEIS Foundation Inc. [3], from the pinned `oeisdata` snapshot at revision `a6e0f22854cc1c307da428e9d6295093781df7fa`, and are used under the Creative Commons Attribution-ShareAlike 4.0 license. Each formalization additionally stores the exact entry digest rather than relying on the repository revision alone.

The five examples below are deliberately not compressed into one table. For readability, write $\text{Loop}(f, k, z) = f^{\circ k}(z)$ for these loop bodies, all of which ignore the loop counter, and abbreviate the Lean namespace `GauthierE2.P` as `P`. The displayed "semantic normal form" is the value established by the proof, not merely a pretty-printing or a finite simplifier run. Each separate Lean listing then shows the source specification, frozen hash and tokens, parsed program AST, recognizer equality, literal theorem type, and the theorem with all three semantic definitions unfolded. All five specifications have offset zero, so the visible domain premise $0 \le \text{Int.ofNat}(\text{position})$ is automatic; we retain it to expose the domain-aware theorem exactly.

A016779: one cubing step. The discovered GSLT program has the readable semantic normal form

$$p_{28}(n) = \mathrm{Loop}(x \mapsto x^3, 1, 1 + 3n) = (3n + 1)^3.$$

Full machine-checked statement and proof: `A016779_candidate28_correct`.

Listing 1: A016779, discovered exclusively by the recurrent-vector arm in world 1, generation 8. The proof ranges over the entire domain, not only the 36 published terms.

```
-- OEIS %N: a(n) = (3*n + 1)^3. %O 0,2
def A016779.source := sourceOf "A016779"
  "56bc1bf16ed408a98b0d997bc8006d1b34e8796dcaf47bfbd91586155f526b97" 0
def A016779.spec := specOf 0 (fun n => (3 * n + 1) ^ 3)

def candidate28 : FrozenCandidate where
  programSha256 := "dba931cee529c62f5c8284b78fbb0974327221981a029c00f99a6b4d0711eb2d"
  tokens := [10, 10, 5, 10, 5, 1, 1, 10, 10, 3, 10, 3, 3, 9]
  program := P.loop (P.mult (P.mult P.X P.X) P.X) P.o
    (P.addi P.o (P.addi (P.addi P.X P.X) P.X))
  recognized := rfl

#check A016779_candidate28_correct :
  CandidateRealizes A016779.spec candidate28

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate28.program
        (Int.ofNat position) =
          some ((3 * Int.ofNat position + 1) ^ 3) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016779.spec, specOf, SequenceSpec.index]
    using A016779_candidate28_correct
```

A016780: two squaring steps. Here the network expresses a fourth power as two iterations of squaring:

$$p_{29}(n) = \mathrm{Loop}(x \mapsto x^2, 2, 1 + 3n) = \left((3n+1)^2\right)^2 = (3n+1)^4.$$

Full machine-checked statement and proof: A016780_candidate29_correct.

Listing 2: A016780, discovered exclusively by the belief-register arm through derived retention in world 1, generation 4. The OEIS entry contains 31 published terms; the theorem is universal.

```
-- OEIS %N: a(n) = (3*n + 1)^4. %O 0,2
def A016780.source := sourceOf "A016780"
  "86c883b0debc37c8ada7ab8f7e0414d0f6f156b2533f6090d993718f45737168" 0
def A016780.spec := specOf 0 (fun n => (3 * n + 1) ^ 4)

def candidate29 : FrozenCandidate where
  programSha256 := "c57882d00078998c452c5feee21dc0c7d0399e0f60379d118fd421c54acf6d67"
  tokens := [10, 10, 5, 2, 1, 1, 2, 3, 10, 5, 3, 9]
  program := P.loop (P.mult P.X P.X) P.tw
    (P.addi P.o (P.mult (P.addi P.o P.tw) P.X))
  recognized := rfl

#check A016780_candidate29_correct :
  CandidateRealizes A016780.spec candidate29

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate29.program
        (Int.ofNat position) =
          some ((3 * Int.ofNat position + 1) ^ 4) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016780.spec, specOf, SequenceSpec.index]
    using A016780_candidate29_correct
```

A016814: one squaring step. The initial expression changes, while the learned loop skeleton is reused:

$$p_{30}(n) = \mathrm{Loop}(x \mapsto x^2, 1, 1 + 4n) = (4n + 1)^2.$$

Full machine-checked statement and proof: A016814_candidate30_correct.

Listing 3: A016814, discovered exclusively by the belief-register arm through derived retention in world 2, generation 1. The proof replaces a 44-term match with a full-domain theorem.

```
-- OEIS %N: a(n) = (4*n + 1)^2. %O 0,2
def A016814.source := sourceOf "A016814"
  "2bb3a661541a70b58c6a0e2fd6b96931502217d3eead53dd16a0d04607b4a3a2" 0
def A016814.spec := specOf 0 (fun n => (4 * n + 1) ^ 2)

def candidate30 : FrozenCandidate where
  programSha256 := "49e6cea5ab07ccf1dfac616e36e42e5e49c7ed074c227c30ddf946f24587efef"
  tokens := [10, 10, 5, 1, 1, 2, 2, 3, 10, 5, 3, 9]
  program := P.loop (P.mult P.X P.X) P.o
    (P.addi P.o (P.mult (P.addi P.tw P.tw) P.X))
  recognized := rfl

#check A016814_candidate30_correct :
  CandidateRealizes A016814.spec candidate30

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate30.program
        (Int.ofNat position) =
          some ((4 * Int.ofNat position + 1) ^ 2) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016814.spec, specOf, SequenceSpec.index]
    using A016814_candidate30_correct
```

A070439: a computed modulus. The program does not contain the literal 16; it constructs it by squaring twice from two:

$$p_{45}(n) = n^2 \bmod \mathrm{Loop}(x \mapsto x^2, 2, 2) = n^2 \bmod 16.$$

Full machine-checked statement and proof: `A070439_candidate45_correct`.

Listing 4: A070439, discovered exclusively by the recurrent-vector arm in world 1, generation 5. The synthesized denominator computes 16 rather than storing it.

```
-- OEIS %N: a(n) = n^2 mod 16. %O 0,3 (101 published terms)
def A070439.source := sourceOf "A070439"
  "df5acb01e6c2570f6b4592782d285292d0cb69a4cd1987052c2f5ac6b0cc6de4" 0
def A070439.spec := specOf 0 (fun n => n ^ 2 % 16)

def candidate45 : FrozenCandidate where
  programSha256 := "315061126a0a0fb9eb5c6b0de9be9daa0977d662244d07020ff5276021a5f81e"
  tokens := [10, 10, 5, 10, 10, 5, 2, 2, 9, 7]
  program := P.modu (P.mult P.X P.X)
    (P.loop (P.mult P.X P.X) P.tw P.tw)
  recognized := rfl

#check A070439_candidate45_correct :
  CandidateRealizes A070439.spec candidate45

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate45.program
        (Int.ofNat position) =
          some (Int.ofNat position ^ 2 % 16) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A070439.spec, specOf, SequenceSpec.index]
    using A070439_candidate45_correct
```

A070478: the cubed companion. The same learned construction of 16 is paired with a cubed numerator:

$$p_{46}(n) = n^3 \bmod \mathrm{Loop}(x \mapsto x^2, 2, 2) = n^3 \bmod 16.$$

Full machine-checked statement and proof: `A070478_candidate46_correct`.

Listing 5: A070478, discovered exclusively by the recurrent-vector arm in world 1, generation 2. Its 96 published terms become a universal stable-evaluation claim.

```
-- OEIS %N: a(n) = n^3 mod 16. %O 0,3
def A070478.source := sourceOf "A070478"
  "cd9f8752f701420ceb43a72e77db95d1d3816fddc6db55c8b531aabbab41947b" 0
def A070478.spec := specOf 0 (fun n => n ^ 3 % 16)

def candidate46 : FrozenCandidate where
  programSha256 := "310209b1db332a56d73abbd2476b62292b37548b20f7114124632ba8cdbb61cc"
  tokens := [10, 10, 5, 10, 5, 10, 10, 5, 2, 2, 9, 7]
  program := P.modu (P.mult (P.mult P.X P.X) P.X)
    (P.loop (P.mult P.X P.X) P.tw P.tw)
  recognized := rfl

#check A070478_candidate46_correct :
  CandidateRealizes A070478.spec candidate46

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate46.program
        (Int.ofNat position) =
          some (Int.ofNat position ^ 3 % 16) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A070478.spec, specOf, SequenceSpec.index]
    using A070478_candidate46_correct
```

Two details in these listings repay attention. The programs need not mirror the definitions syntactically: A070439's specification is $n^2 \bmod 16$, while the discovered program computes the modulus as `loop (mult X X) tw tw`—sixteen obtained by iterated squaring from two, rather than written down—and the proof must reconcile the two. And the discovering arms are not the ones the headline count favours: two of these five came from the belief-register architecture, which finished *last* on raw yield (294 against the recurrent vector's 435). All five were found by exactly one architecture. The diversity argument of §8.2 was until now a statement about counts; here it is a statement about programs that are provably correct.

What this does and does not establish. The cohort was selected for formalizability, so 33/33 is emphatically not an estimate of the correctness rate among all 984 discoveries; the sequences resting on three or four published terms are exactly those excluded by that selection, and they remain the open question. More subtly, the proofs certify that each program equals *our formalization* of a definition. Whether that formalization faithfully renders the encyclopedia's source text is a separate judgment, and it is not one a kernel can discharge. We therefore keep two links distinct: program ≡ specification is factive; specification ≡ intended sequence is a reviewed translation carrying its own provenance. Because the proof depends on the translation, the two must not be pooled as independent support—an independent semantic review is a genuinely sepa-

rate source, and only that can move the second link. This dependence boundary is formalized by `translation_proof_have_no_additiveRevisionLicense`.

Refutation is generative. A failed adjudication is not a discarded result. A formalized definition is a certified generator: it produces additional terms beyond the published window, with proof, turning a weak finite match into a targeted counterexample and a localized repair obligation—the first index at which the program and the specification provably differ. Ten of our 984 also cover entries the encyclopedia marks `dead`: nine are duplicate identifiers, which are redirected to canonical entries, while one is an erroneous historical version and must not be counted as a current target. Neither case is evidence that a candidate computes the intended canonical sequence. Together these convert the discovery ledger from a count into a graded evidential structure: finite term agreement, keyword and reference-program corroboration, reviewed formalization, and finally factive proof or factive counterexample.

10 From leakage to conjecture

Near-equivalence is dual to conjecture: what threatens a benchmark as contamination is, read the other way, a discovered near-identity. The sharpest instance is the pairing of the *fastest* and the *smallest* candidate covering an entry. These minimize opposite terms of Levin's Kt complexity, $Kt(x) = \min_p |p| + \log t(p)$ [5]: program length and running time. Two unlike programs converging on the same observed terms motivate an equivalence conjecture, but they provide independent evidence only when their search lineages are source-disjoint. Otherwise their common training corpus or ancestral discoveries remain shared causes and additive revision is not licensed. Even with independent provenance, finite agreement is corroboration rather than entailment. *Proving* the two programs equivalent on all n converts finite-prefix agreement into total agreement; the induction predicates such proofs require are exactly what recent conjecturing systems learn to generate [2]. Our verified layer offers to discharge them as kernel-checked theorems rather than trusted external calls. The interesting conjectures live where extensional similarity is high but *syntactic* intensional similarity is low—two unlike programs apparently computing one sequence—and shared formally proved properties become lemmas toward the identity proof.

11 Status and roadmap

The verified DSL semantics, recognizer, big-step modalities, and the three central obligations are machine-checked and axiom-clean. The synthesizer's data pipeline, exact split gate, and the WM-PLN extensional and intensional layers are implemented and measured over the current corpus, with the numbers reported above; the self-learning loop of Section 8 compounds under three registered replay settings across three worlds. Thirty-three frozen discoveries now additionally have full-domain correctness proofs against pinned formal specifications. Near-term work is to (i) independently review the 33 source-to-Lean translations; (ii) adjudicate a cohort selected from the shortest, weakest-evidence entries, where counterexamples are most likely; (iii) prove the Python abstract interpreter sound against the Lean evaluator before using its conclusions for hard pruning; (iv) bind the Lean canonical policy to exported operator-table metadata; and (v) retain the fastest and smallest program per sequence for provenance-aware equivalence conjectures. The unifying aim is an OEIS synthesis loop whose catalogue coverage, source interpretation, and full-domain program claims are explicitly distinguished and independently auditable.

References

[1] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.

[2] T. Gauthier and J. Urban. *Learning Conjecturing from Scratch*. arXiv:2503.01389, 2025.

[3] OEIS Foundation Inc. *The On-Line Encyclopedia of Integer Sequences* and the `oeisdata` repository. `https://oeis.org`. Data snapshot: `https://github.com/oeis/oeisdata/tree/a6e0f22854cc1c307da428e9d6295093781df7fa`. OEIS content is licensed CC BY-SA 4.0.

[4] P. Cousot and R. Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL 1977. `https://doi.org/10.1145/512950.512973`.

[5] L. A. Levin. *Universal Sequential Search Problems*. Problems of Information Transmission, 9(3):265–266, 1973. `https://www.mathnet.ru/eng/ppi914`.

[6] B. Goertzel, M. Iklé, I. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009. `https://doi.org/10.1007/978-0-387-76872-4`.